

R Programming

#02 Basic R

Outline

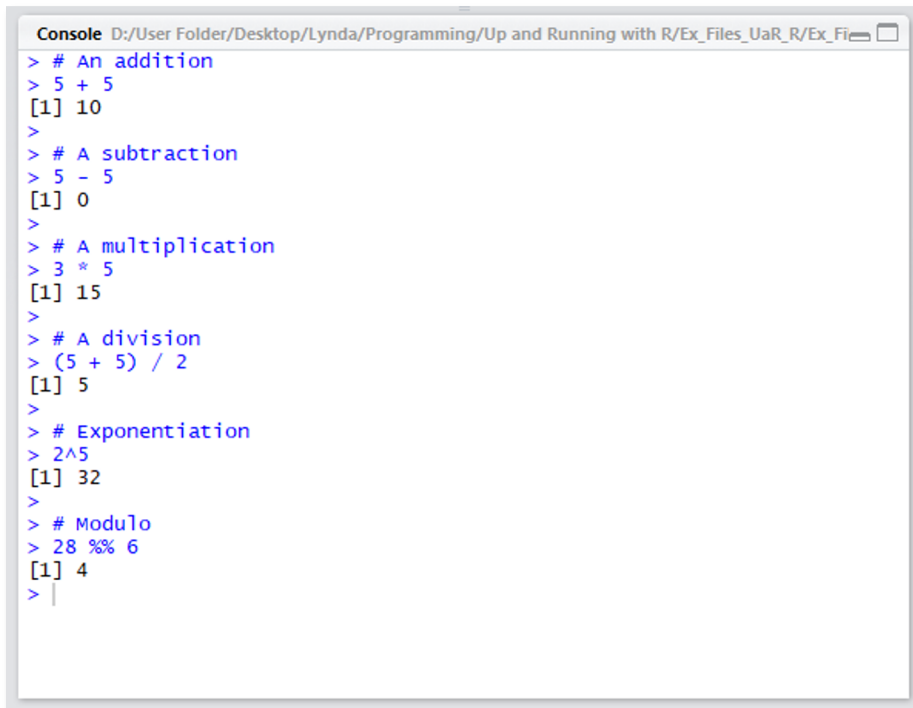
- R Basics
- Packages in R
- Search Help in R
- Functions in R
- Control Structures in R
- Import/Export in R

#02

R BASICS

Arithmetic with R

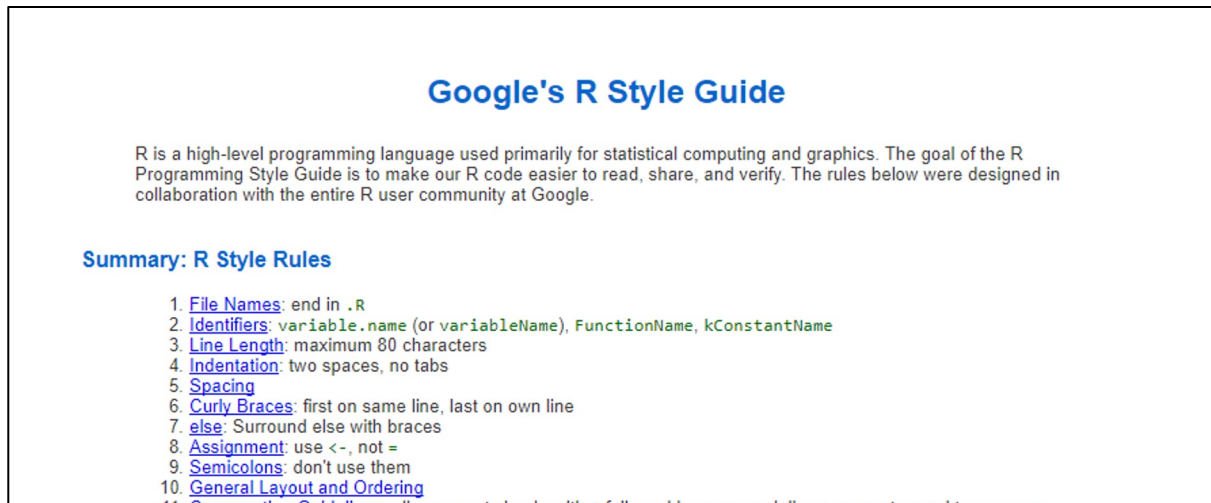
- In the most basic form, R can be used as a simple calculator.
- Using following arithmetic operators:
 - Addition: +
 - Subtraction: -
 - Multiplication: *
 - Division: /
 - Exponentiation: ^
 - Modulo: %%



```
Console D:/User Folder/Desktop/Lynda/Programming/Up and Running with R/Ex_Files_UaR/Ex_Fi
> # An addition
> 5 + 5
[1] 10
>
> # A subtraction
> 5 - 5
[1] 0
>
> # A multiplication
> 3 * 5
[1] 15
>
> # A division
> (5 + 5) / 2
[1] 5
>
> # Exponentiation
> 2^5
[1] 32
>
> # Modulo
> 28 %% 6
[1] 4
> |
```

Variable Assignment in R

- In R, variable assignment is done using either “<-” or “=”
- For example: “**x<-4**” is equivalent to “**x=4**”
- However, some style guide such as Google R Style Guide prefers the use of “<-” instead of “=”



The screenshot shows the title "Google's R Style Guide" in blue. Below it is a paragraph explaining that R is a high-level programming language used for statistical computing and graphics, and that the goal of the style guide is to make R code easier to read, share, and verify. Below this is a section titled "Summary: R Style Rules" in blue, followed by a numbered list of 10 rules. The rules are: 1. File Names: end in .R; 2. Identifiers: variable.name (or variableName), FunctionName, kConstantName; 3. Line Length: maximum 80 characters; 4. Indentation: two spaces, no tabs; 5. Spacing; 6. Curly Braces: first on same line, last on own line; 7. else: Surround else with braces; 8. Assignment: use <-, not =; 9. Semicolons: don't use them; 10. General Layout and Ordering.

Google's R Style Guide

R is a high-level programming language used primarily for statistical computing and graphics. The goal of the R Programming Style Guide is to make our R code easier to read, share, and verify. The rules below were designed in collaboration with the entire R user community at Google.

Summary: R Style Rules

1. File Names: end in .R
2. Identifiers: variable.name (or variableName), FunctionName, kConstantName
3. Line Length: maximum 80 characters
4. Indentation: two spaces, no tabs
5. Spacing
6. Curly Braces: first on same line, last on own line
7. else: Surround else with braces
8. Assignment: use <-, not =
9. Semicolons: don't use them
10. General Layout and Ordering

<https://google.github.io/styleguide/Rguide.xml>

Basic Data Types in R

- Some of the basic data types in R are:
 - **Numeric:**1, 10, 15, 1C, 2E,.....
 - **Integer:**2, 15, 33, 13, 18,.....
 - **Logical:** TRUE or FALSE
 - **Characters:** Text or strings
- The data types of a variable can be checked with **class()** function.

```
Console D:/User Folder/Desktop/Lynda/Programming/Up and Running with R/Ex_Files_UaR/Ex_Fi
> my_numeric <- 42
> my_character <- "universe"
> my_logical <- FALSE
>
> # Check class of my_numeric
> class(my_numeric)
[1] "numeric"
>
> # Check class of my_character
> class(my_character)
[1] "character"
>
> # Check class of my_logical
> class(my_logical)
[1] "logical"
>
```

Vectors in R

- Vectors are **one-dimension arrays** that can hold numeric, integer, character or logical data.
- In R, vectors are created using function **c()** ,for example:
 - `numeric_vector <- c(1, 10, 49)`
 - `character_vector <- c("a", "b", "c")`
 - `boolean_vector <- c(TRUE, FALSE, TRUE)`
- In R, we can set custom name for the vector. Example:
 - `names(character_vector) <- c("One", "Two", "Three")`

```
Console D:/User Folder/Desktop/Lynda/Programming/Up and Running with R/Ex_Files_UaR_R/Ex_Files_UaR_R/Chapter06/06_01.R
> # Vectors
> numeric_vector <- c(1, 10, 49)
> character_vector <- c("a", "b", "c")
> boolean_vector <- c(TRUE, FALSE, TRUE)
> names(character_vector) <- c("One", "Two", "Three")
> character_vector
  One   Two Three 
"a"   "b"   "c"
```

Arithmetic of Vectors in R

- Vectors can perform **element-wise arithmetic operation**.
- Sum and average of the elements in the vector can be calculated using **`sum(vector)`** and **`mean(vector)`** respectively.
- Vectors can be used to do comparison, for example:
 - `vector = c(1,2,3,4,5)`
`vector > 3`
Output:
`FALSE FALSE FALSE TRUE TRUE`
- Element in specific position in the vector can be selected using **`vector[position]`**.
 - `Bool_value = c(TRUE, FALSE, TRUE, FALSE)`
`Vector = c(2, 3, 5, 6)`
`Vector[Bool_value]`
Output:
`2 5`

Factors in R

- Factors are **categorical variables** that are useful in summary statistics, plots and regressions.
- To create factors in R, use **factor()** function.
- The factor levels can be changed using the **levels()** function.

```
my_vector <- c("L", "S", "L", "M", "M")

# Option 1
my_factor <- factor(my_vector)
levels(my_factor) <- c("Large", "Medium", "Small")

# Option 2
my_factor <- factor(my_vector,
                    levels = c("S", "M", "L"),
                    labels = c("Small", "Medium", "Large"))
```

Matrices in R

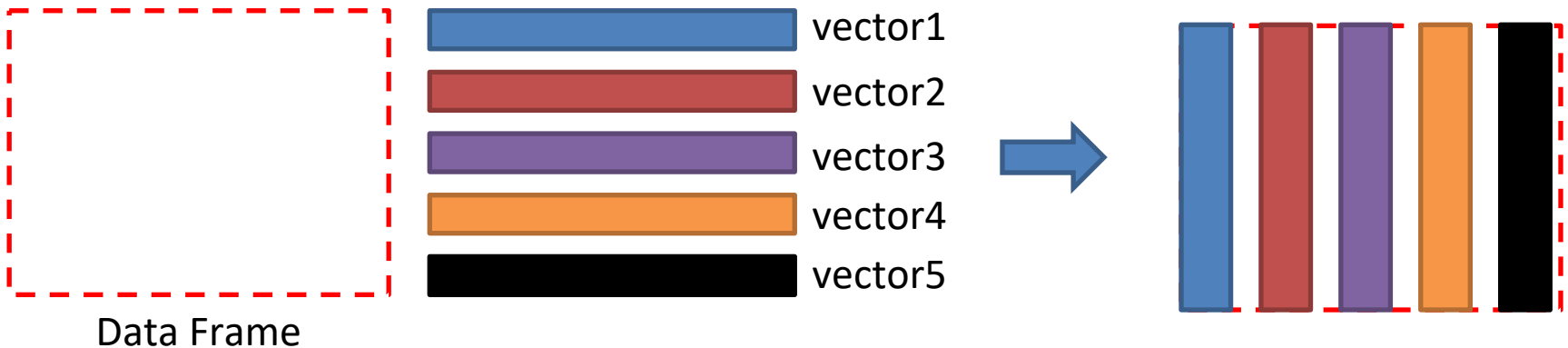
- In R, a matrix is a **collection of elements** of the same data type (numeric, character, or logical) arranged into a fixed number of **rows** and **columns**.
- Since we are only working with rows and columns, a matrix is called **two-dimensional**.
- You can construct a matrix in R with the **matrix()** function. Consider the following example:
 - `matrix(1:9, byrow = TRUE, nrow = 3)`
- The row and column of the matrix can be named using **rownames()** and **colnames()** respectively.
 - Example: `colnames(matrix) <- names`

Operations in Matrices

- We can calculate the column sum and row sum using **colSums()** and **rowSums()** respectively.
 - Example: **colSums(matrix)**
- Additional data can be bind into the matrix using **cbind()** and **rbind()**.
 - **cbind(matrix_A, matrix_B)**: add matrix_B into the “right” side of matrix_A
 - **rbind(matrix_A, matrix_B)**: add matrix_B into the “bottom” side of matrix_A
- Element in specific position in the matrix can be selected using **matrix[row_position, col_position]**.

Creating a Data Frame in R

- In R, data in matrix should consists of same data type. However, if the data consists of **different data types**, a **data frame** is more suitable for this case.
- Data frame can be constructed with the **data.frame()** function.
 - Example: `data.frame(vector1, vector2, vector3, vector4, vector5)`
- Different vectors will be passed as argument and they will become the **different columns of your data frame**.
- Because every column has the same length, the vectors you pass must also have the **same length**.



Data Frame in R

- When dealing with huge data frame, we can use **head(data frame)** or **tail(data frame)** to show only the first few observations or last few observations of the data frame respectively.
- In order to inspect the structure of the data frame, we can use the function **str()**

```
R Console
> str(mtcars)
'data.frame':  32 obs. of  11 variables:
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
 $ disp: num  160 160 108 258 360 ...
 $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
 $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
 $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
 $ qsec: num  16.5 17 18.6 19.4 17 ...
 $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
 $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
 $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
 $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
> |
```

Accessing Data Frame

- Data frame can be access via **dataframe[row position, column position], dataframe[row position, column name]**
- Column of the data frame also can be assess by using “\$”
 - For example: **dataframe\$column1**
- Subset of the data frame can be obtained through function **subset(dataframe, subset=condition)**
- The data frame can be sorted using **order()** function.
 - For example: **order(dataframe\$column)**

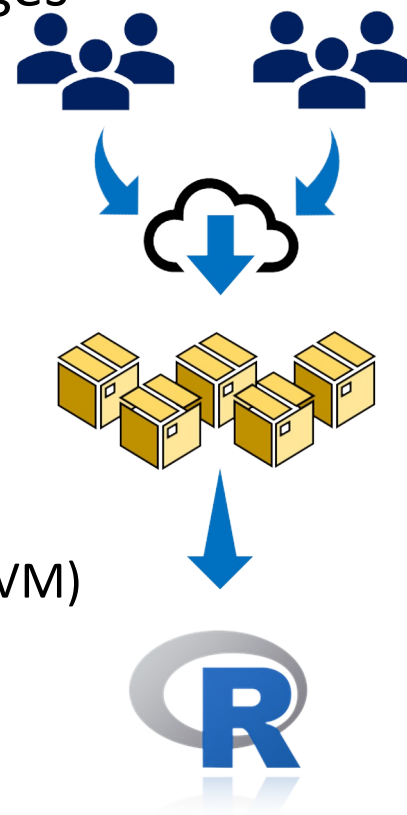
Characters Data in R

- Character's data in R often use **double quotes**.
- **Concatenation** of strings can be done using **paste()** function. For example: `paste(string1, string2, sep="")`
- To print/display output on the console window, use **print()** function.

PACKAGES IN R

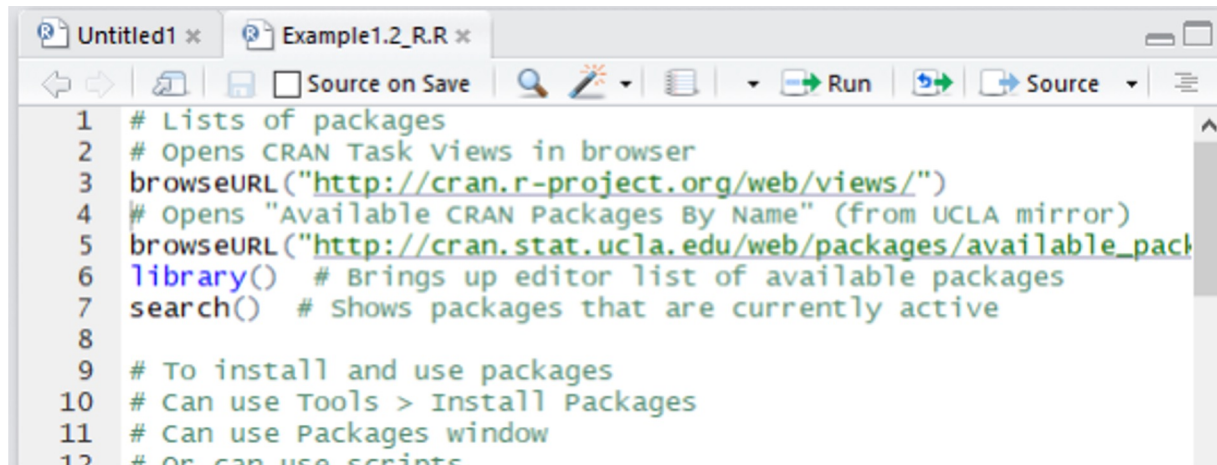
R Packages

- R is highly extensible with widely available packages contributed by community.
- Easy sharing of scripts.
- Comprehensive R Archive Network (CRAN)
- Currently lists over 10000 free packages for R
- Example of packages such as:
 - ggplot2 – for advanced visualization of data
 - e1071 – standard library for support vector machine (SVM)
- To manage R packages, there are two ways:
 - via R Console
 - via RStudio



List available R Packages

- Line 3 code will open up the URL in web browser which shows a large list of categories of packages that are available.
- Line 5 code will shows a long list of packages by name.
- To show the installed packages in the machine, use **library()**
- To show the current active packages, use **search()**



```
1 # Lists of packages
2 # Opens CRAN Task Views in browser
3 browseURL("http://cran.r-project.org/web/views/")
4 # opens "Available CRAN Packages By Name" (from UCLA mirror)
5 browseURL("http://cran.stat.ucla.edu/web/packages/available_pack
6 library() # Brings up editor list of available packages
7 search() # Shows packages that are currently active
8
9 # To install and use packages
10 # Can use Tools > Install Packages
11 # Can use Packages window
12 # Or can use scripts
```

Install R Packages

- There are several ways to **install** R packages in RStudio.
 - Method 1: Using function `install.packages()`
 - For example: `install.packages("package_name")`
 - Method 2: Install package through RStudio GUI (online/manual)
 - Can be accessed through:
 - Package pane at the bottom right and click install.
 - Tools menu > Install Packages
- To **load** the installed packages, use:
 - `library("package_name")`
 - `require("package_name")`
- To bring up the **documentation** of the package:
 - `library(help="package_name")`

Update, Detach and Delete R Packages

- To update packages:
 - Use function: **update.packages()**
 - Or through Tools > Checks for Updates
- To unload/detach an active package:
 - Use function: **detach("package:package_name", unload=TRUE)**
 - Untick the function that want to detach at Package pane.
- To delete an installed package:
 - Use function: **remove.packages("package_name", lib="~/R/win-library/3.4")**
 - Click the "x" at the right of the list of installed package in the Package pane.

SEARCH HELP IN R

Vignettes in R

- Most of the R packages have a list of vignettes, which consist of the examples and detail description regarding the package.
- To bring up the list of vignettes of the package:

```
vignette(package = "package_name")
```

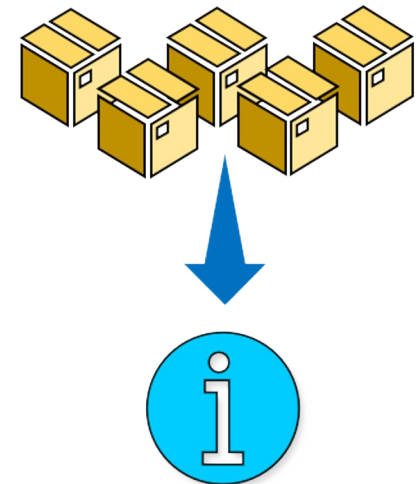
- To show an interactive list of vignettes of the package in HTML through browser:

```
browseVignettes(package = "package_name")
```

- To show all vignettes for all available packages:

```
vignette()
```

```
browseVignettes()
```



FUNCTIONS IN R

Define a function

- To create a “trigger” for a series of action in a more manageable way.
- Note: the R script must be loaded first before the function can be called in the R Console.

Script.R

```
myFuncName <- function() {  
  print("Hello")  
}
```

Function name

Parameters

```
myFunc2Name <- function(param1, param2)  
{  
  print(param1+param2)  
}
```

Function name

```
myFunc3Name <- function(param1) {  
  myFunc2Name(param1, 10)  
  print(param1)  
}
```

R Console

```
> myFuncName()  
[1] Hello  
> myFunc2Name(15, 4)  
[1] 19  
> myFunc3Name(25)  
[1] 35  
[1] 25
```


CONTROL STRUCTURE IN R

Control Structures in R

- If.....Else in R

```
if(condition) {  
    #statement  
}
```

- For loop in R

```
for(x in 1: 10) {  
    print(x)  
}
```

- While loop in R

```
x <- 1;  
while(x<11) {  
    print(x)  
    x = x+1;  
}
```

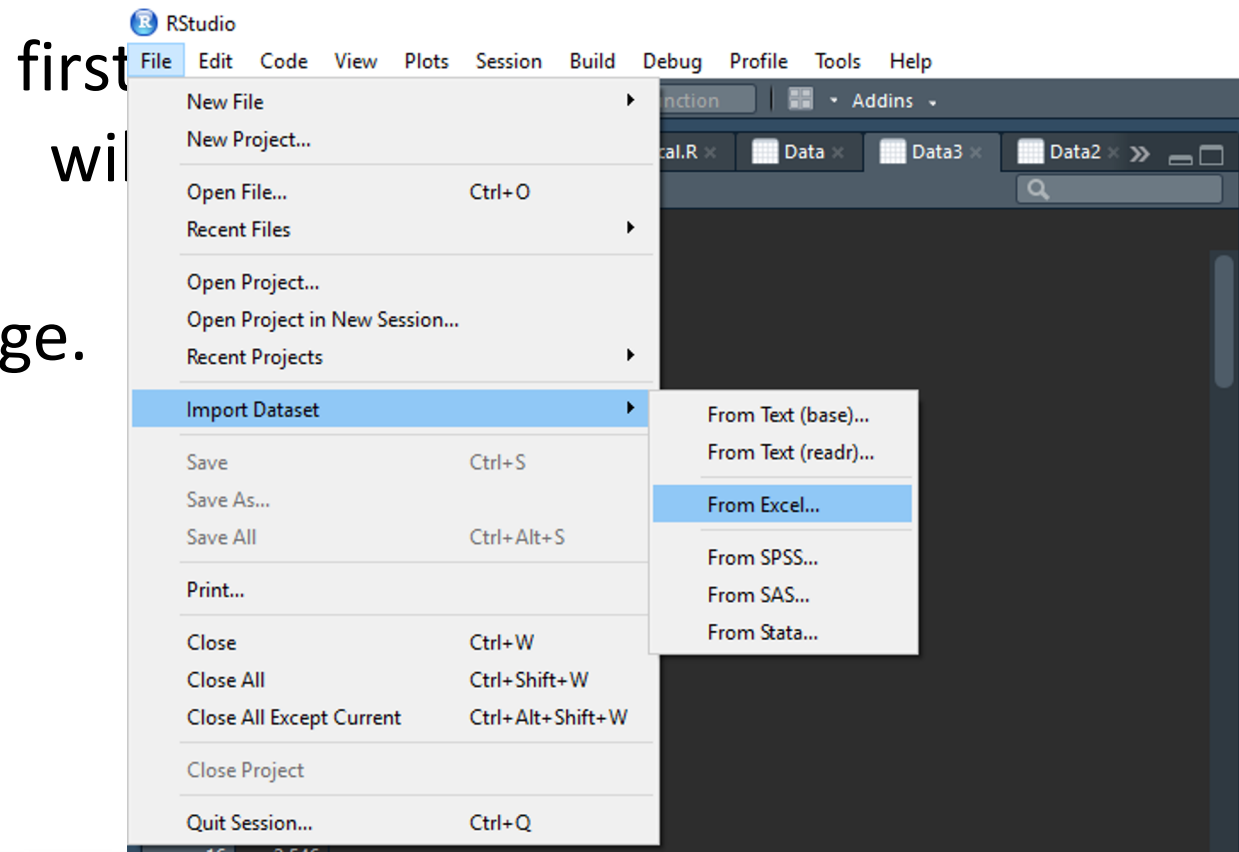
IMPORT/EXPORT IN R

Save/Load and Import/Export

- In R, data can be save into R binary data (.Rdata extension), which only readable by R.
- This can be done by **save()** function.
 - For example: `save(variable, file="path/filename.Rdata")`
 - The data saved in Rdata format can be loaded into R using **load()**
- In R, data also can be exported to a Excel-readable format (.csv) using **write.csv()**
 - For example: `write.csv(dataframe, "path/filename.csv")`
 - The CSV file can be imported into R using **read.csv()**

Import from Excel

- In R, Excel file can be imported by navigating to the option pane: File → Import Dataset → From Excel
- If it will download install a package.



Import from Excel

- File selection window will pop out as follow:

Import Excel Data

File/Url:
 Browse...

Data Preview:

Staff (double)	Position (character)	Blood Type (character)	Weight (double)	Height (double)	Qualification (character)
1	Senior Lecturer	A		60	172
2	Lecturer	B		55	164
3	Professor	O		65	163
4	Associate Professor	AB		70	166
5	Associate Professor	O		61	167
6	Senior Lecturer	O		58	162
7	Senior Lecturer	B		48	172
8	Lecturer	A		68	161
9	Lecturer	A		55	169
10	Associate Professor	AB		62	158
11	Professor	O		58	175
12	Senior Lecturer	O		58	156
13	Senior Lecturer	B		48	154
14	Senior Lecturer	O		58	169
15	Senior Lecturer	B		48	160
16	Lecturer	A		68	178
17	Lecturer	A		55	156
18	Tutor	A		55	174
19	Professor	O		58	167
20	Senior Lecturer	O		58	154
21	Senior Lecturer	B		48	158
22	Lecturer	A		68	160

Previewing first 50 entries.

Import Options:

Name: Max Rows: ☒ First Row as Names

Sheet: Skip: ☒ Open Data Viewer

Range: NA:

Code Preview:

```
library(readxl)
Data <- read_excel("C:/Users/steph/Desktop/Data.xlsx")
View(Data)
```

Import Cancel

Uncheck this if the data file does not has header

Click Import once done

The variable name that data imported to